Space Objects

- position: Position
- id: string
- currentTime: int

+ SpaceObject(pos: Position, identifier: string)
+ getPosition(): Position
+ getId(): string
+ status(): void (pure virtual)
+ interact(other: shared_ptr<SpaceObject>): void (pure virtual)
+ getCurrentPosition(): Position (pure virtual)
+ update(): void (pure virtual)
+ advanceTime(): void
+ getType(): string (pure virtual)
+ ~SpaceObject(): (default)

Static Methods:
+ validateId(identifier: string): string (private)

Spaceship

- state: SpaceshipState
- method: flightMethod
- destination: Position
- direction: Direction
- speed: float
- sites: vector<shared_ptr<SpaceObject>>
- sitesMap: unordered_map<Position, shared_ptr<SpaceObject>>
- sitesDestinations: unordered_map<string, Position>

+ Spaceship(pos: Position, identifier: string, sites: vector<shared_ptr<SpaceObject>>)
+ setSpeed(newSpeed: float): void
+ getSpeed(): float
+ setState(newState: SpaceshipState): void
+ setDestinationByPosition(dest: Position): void
+ getCurrentPosition(): Position (override)
+ isAlive(): bool
+ move(newPosition: Position): void (virtual)
+ stop(): void (virtual)
+ interact(other: shared_ptr<SpaceObject>): void (override)
+ update(): void (override)
+ status(): void (override)
+ getType(): string (override, pure virtual)
+ setCourse(angle: float): void
+ printState(typeName: string): void
+ printFlightMethod(typeName: string): void
+ setSiteDestination(siteName: string): void
+ calculateCoursePosition(): Position
+ calculateNewPosition(): Position
+ getState(): SpaceshipState
+ ~Spaceship(): (override, default)

Static Methods:
+ logCreation(spaceshipType: string, identifier: string, speed: float): void

SpaceStation

- crystalsAmount: int
- productionRate: int
- dockingShuttles: vector<shared_ptr<SpaceObject>>

+ SpaceStation(crystalsAmount: int, productionRate: int, position: Position, identifier: string)
+ status(): void (override)
+ update(): void (override)
+ getCurrentPosition(): Position (override)
+ unloadCrystals(): int
+ interact(other: shared_ptr<SpaceObject>): void (override)
+ getType(): string (override)
+ filterDockingShuttles(): vector<shared_ptr<SpaceObject>>
+ printDockingShuttles(): void

FortressStar

- crystalAmount: int
- dockingShuttles: vector<shared_ptr<SpaceObject>>

+ FortressStar(crystalAmount: int, position: Position, identifier: string)
+ status(): void (override)
+ interact(other: shared_ptr<SpaceObject>): void (override)
+ getCurrentPosition(): Position (override)
+ update(): void (override)
+ addCrystals(amount: int): void
+ getType(): string (override)

Position

- x: float
- y: float
- UNIT_TO_KM: float (static, constexpr)

+ Position(x: float = 0.0, y: float = 0.0)
+ Position(Position const& Other) (default)
+ Position(Position&& other) noexcept (default)
+ operator=(other: Position const&) -> Position& (default)
+ operator=(other: Position&&) noexcept -> Position& (default)
+ ~Position() (default)
+ getX(): float
+ getY(): float
+ setX(x1: float): void
+ setY(y1: float): void
+ move(x1: float, y1: float): void
+ distance(pos1: Position, pos2: Position): float (static)
+ distance(x1: float, y1: float): float
+ operator==(position: Position): bool
+ operator<<(os: ostream, position: Position): ostream& (friend
+ toString(): string

Static Methods (Specializations):
+ hash<Position>::operator()(pos: Position): size_t

Shuttle

- crystalsContainers: int
- maxCrystalsContainers: int
- powerUnits: int
- maxPowerUnits: int
- fortress: Position
- station: Position
- fortressStar: shared_ptr<FortressStar>
- spaceStation: shared_ptr<SpaceStation>
- supplyMissions: queue<pair<shared_ptr<SpaceStation>, shared_ptr<FortressStar>>>
- pilot: shared_ptr<Midshipman>
- finishedMission: bool
- needToWait: bool

+ Shuttle(pos1: Position, identifier1: string, speed: float, pilot: shared_ptr<Midshipman>, sites: vector<shared_ptr<SpaceObject>>)
+ update(): void (override)
+ interact(other: shared_ptr<SpaceObject>): void (override)
+ loadCrystals(st: shared_ptr<SpaceStation>): void
+ startSupplyMission(): void
+ finishSupplyMission(): void
+ decreaseDefPowerUnits(): void
+ getDefPowerUnits(): int
+ unloadCrystals(): void
+ addSupplyMission(mission: pair<shared_ptr<SpaceStation>, shared_ptr<FortressStar>>): void
+ getType(): string (override)
+ status(): void (override)

StarDestroyer

- pilot: shared_ptr<Admiral>
- destroyerMissiles: vector<string>
- missilesMap: unordered_map<string, shared_ptr<Missile>>
- missilesCounter: int

+ StarDestroyer(pos: Position, identifier: string, speed: float = DEFAULT_STAR_DESTROYER_SPEED, pilot: shared_ptr<Admiral>, sites: vector<shared_ptr<SpaceObject>>)
+ status(): void (override)
+ missileUpdate(falconsPositions: vector<pair<string, Position>>): string
+ shoot(targetPos: Position): void
+ getMissilesNameAndPosition(): vector<pair<string, Position>>
+ getType(): string (override)

Private Methods:
- createMissile(targetPos: Position): string

Direction

- angle: float

+ Direction(angle1: float)
+ Direction(src: Position, dst: Position)
+ getAngle(): float
+ degreesToRadians(degrees: float): float (static)
+ getAngleInRadians(): float
+ setAngle(newAngle: float): void
+ calculateAngle(src: Position, dst: Position): float (static)

Falcon

- attPowerUnits: int
- maxAttPowerUnits: int

+ Falcon(pos: Position, identifier: string, patrolSites: vector<shared_ptr<SpaceObject>> = {})
+ interact(other: shared_ptr<SpaceObject>): void (override)
+ killFalcon(): void
+ getAttPowerUnits(): int
+ increaseAttPowerUnits(): void
+ decreaseAttPowerUnits(): void
+ canAttack(attackerPower: int, targetDefense: int, targetPos: Position, distanceToTarget: float, closetBomber: float): bool
+ attack(target: shared_ptr<Shuttle>, distanceToTarget: float, closetBomber: float): void
+ getType(): string (override)

Timer

- currentTick: int (static)
- tickInterval: int (static)

+ initialize(interval: int): void (static)
+ getCurrentTick(): int (static)
+ advanceTick(): void (static)

Missile

- target: Position
- distance: float
- isDestroyed: bool

+ Missile(destroyer: Position, targetPos: Position, identifier: string, sites: vector<shared_ptr<SpaceObject>>)
+ status(): void (override)
+ updateAndCheckTarget(falconsPositions: vector<pair<string, Position>>): string
+ isMissileDestroyed(): bool
+ getTarget(): Position
+ getType(): string (override)
+ ~Missile(): (override, default)

TIEBomber

- pilot: shared_ptr<Commander>
- visitedSites: unordered_set<shared_ptr<SpaceObject>>
- closestSite: shared_ptr<SpaceObject>
- firstSite: shared_ptr<SpaceObject>

+ TIEBomber(pos: Position, identifier: string, speed: float = DEFAULT_TIE_BOMBER_SPEED, pilot: shared_ptr<Commander>, sites: vector<shared_ptr<SpaceObject>>)
+ update(): void (override)
+ interact(other: shared_ptr<SpaceObject>): void (override)
+ getType(): string (override)

Private Methods:
- findClosestSpaceSite(): void
- alpabeticalOrderCase(site: shared_ptr<SpaceObject>): shared_ptr<SpaceObject>
- flyToClosestSpaceSite(): void

Model

- objectsMap: unordered_map<string, shared_ptr<SpaceObject>>
- agents: unordered_map<string, shared_ptr<ImperialAgent>>
- ids: vector<string>
- uniqueIds: unordered_set<string>
- starDestroyersSpaceships: vector<string>
- missiles: vector<string>
- falcons: vector<string>
- sites: vector<string>
- sitesNumber: unsigned int

+ getInstance(): Model& (static)
+ calculateDistance(pos1: Position, pos2: Position): float (static)
+ addSite(command: vector<string>): void
+ create(command: vector<string>): void
+ statusCMD(): void
+ goCMD(): void
+ attackCMD(command: vector<string>): void
+ shootCMD(command: vector<string>): void
+ stopCMD(command: vector<string>): void
+ positionCMD(command: vector<string>): void
+ destinationCMD(command: vector<string>): void
+ courseCMD(command: vector<string>): void
+ defaultSite(): void
+ SupplyCMD(command: vector<string>): void
+ statusByObj(command: vector<string>): void
+ validateObjectExists(objectName: string): bool
+ validateObjectExists(objectName: string, role: string): bool
+ findClosetBomber(attackerPos: Position): float
+ getPositions(): vector<pair<string, Position>>
+ createFalcon(command: vector<string>, name: string, patrolSites: vector<shared_ptr<SpaceObject>>): void
+ createSpaceshipWithPilot(command: vector<string>, type: string, name: string, patrolSites: vector<shared_ptr<SpaceObject>>): void
+ createDestroyer(pilot: shared_ptr<ImperialAgent>, pos: Position, name: string, patrolSites: vector<shared_ptr<SpaceObject>>): void
+ createBomber(pilot: shared_ptr<ImperialAgent>, pos: Position, name: string, patrolSites: vector<shared_ptr<SpaceObject>>): void
+ createShuttle(pilot: shared_ptr<ImperialAgent>, pos: Position, name: string, patrolSites: vector<shared_ptr<SpaceObject>>): void
+ createTroops(command: vector<string>): void
+ createSpaceship(command: vector<string>): void
+ validateAndInsertId(name: string): bool
+ createAgent(agentType: string, agentName: string, factory: ConcreteImperialAgentFactory): void
+ logAndReportError(errorMessage: string): void (static)
+ createSite(type: string, crystals: int, production: int, pos: Position, name: string): unique_ptr<SpaceObject> (static)
+ printStatusByType(title: string, names: vector<string>, types: unordered_set<string>): void
+ getObject(objName: string): shared_ptr<SpaceObject>
+ clearAllStationsAndFortresses(): void

Private Methods:
- advanceTime(): void
- starDestroyerUpdate(): void
- collectPatrolSites(): vector<shared_ptr<SpaceObject>>

ImperialAgent

- name: string

+ ImperialAgent(agent_name: string)
+ getName(): string
+ ~ImperialAgent(): (virtual, default)
+ operator<<(os: ostream, agent: ImperialAgent): ostream& (friend)

Admiral

- name: string

+ Admiral(name: string)

Commander

- name: string

+ Commander(name: string)

Midshipman

- name: string

+ Midshipman(name: string)

ImperialAgentFactory

+ createMidshipman(name: string): shared_ptr<ImperialAgent>
+ createCommander(name: string): shared_ptr<ImperialAgent>
+ createAdmiral(name: string): shared_ptr<ImperialAgent>

ConcreteImperialAgentFactory

+ createMidshipman(name: string): shared_ptr<ImperialAgent>
+ createCommander(name: string): shared_ptr<ImperialAgent>
+ createAdmiral(name: string): shared_ptr<ImperialAgent>